

Explicacion detallada del programa

1. Proposito general

Este programa toma una imagen en memoria y permite aplicar transformaciones basicas sobre ella:

- mover la imagen a la izquierda, derecha, arriba o abajo;
- rotarla 90 grados;
- deshacer y rehacer acciones;
- repetir la ultima accion;
- reproducir toda la secuencia de acciones desde el inicio.

El objetivo no es construir una interfaz grafica, sino generar archivos PNG por cada paso. De esta manera se puede observar como cambia la imagen despues de cada operacion.

La clase principal es `moving_image`, que se encuentra en `moving_image.h`. El archivo `test.cpp` se usa solo como programa de prueba para ejecutar varias acciones y verificar que todo funcione correctamente.

2. Archivos que participan

2.1. `basics.h`

En este archivo se definen las constantes y recursos base del programa:

- `H_IMG` y `W_IMG`: alto y ancho del lienzo.
- `INIT_Y` e `INIT_X`: posicion donde se inserta la imagen inicial.
- `DEFAULT_R`, `DEFAULT_G` y `DEFAULT_B`: color de fondo.
- `svpng.inc`: biblioteca utilizada para guardar archivos PNG.
- `small_mario_rgb.h`: imagen de Mario almacenada en matrices RGB.

2.2. `moving_image.h`

Este archivo contiene la clase completa. Aqui se implementan:

- la reserva y liberacion de memoria;
- los movimientos y la rotacion;
- el historial de acciones;
- la generacion de archivos PNG.

2.3. `test.cpp`

Es el archivo de prueba. Ejecuta una secuencia automatica de operaciones para comprobar el comportamiento del programa.

3. Representacion de la imagen

La clase `moving_image` guarda la imagen usando seis matrices dinamicas:

- `capa_roja`
- `capa_verde`
- `capa_azul`
- `base_roja`
- `base_verde`
- `base_azul`

Las tres primeras corresponden al estado actual de la imagen. Las otras tres guardan una copia del estado inicial, lo que permite volver al inicio cuando se llama a `repeat_all()`.

Cada matriz es de tipo `unsigned char**`. De esta forma, la imagen se maneja como un arreglo bidimensional de pixeles.

La idea es simple:

- cada pixel tiene tres componentes: rojo, verde y azul;
- cada componente se almacena en una matriz separada;
- al dibujar la imagen, se recorren las tres matrices al mismo tiempo.

4. Construcción del objeto

Cuando se crea un objeto `moving_image`, el constructor realiza estas acciones:

1. Reserva memoria en el *heap* para las seis matrices.
2. Rellena toda la imagen con el color de fondo por defecto.
3. Copia la imagen de `small_mario_rgb.h` hacia el centro del lienzo.
4. Guarda una copia de ese estado en las matrices `base_*`.

Gracias a esto, el objeto siempre comienza con una imagen visible y lista para aplicar transformaciones.

5. Reserva y liberación de memoria

Como la imagen es grande, el programa necesita manejar memoria dinámica.

5.1. `asignar_memoria()`

Crea el arreglo de punteros y luego reserva memoria fila por fila.

5.2. `liberar_memoria()`

Hace el proceso contrario: elimina cada fila y luego libera el arreglo principal.

5.3. Destructor `~moving_image()`

Llama a `liberar_memoria()` para evitar fugas de memoria cuando el objeto termina de usarse.

6. Funcionamiento de los movimientos

Las funciones públicas son:

- `move_left(int d)`
- `move_right(int d)`
- `move_up(int d)`
- `move_down(int d)`

Estas llaman a dos métodos internos:

- `trasladar_horizontal(int d, bool es_historial = false)`
- `trasladar_vertical(int d, bool es_historial = false)`

6.1. Movimiento horizontal

`trasladar_horizontal` mueve los pixeles en el eje X.

El comportamiento es circular:

- si la imagen se mueve demasiado hacia la derecha, vuelve a aparecer por la izquierda;
- si se mueve hacia la izquierda, vuelve a aparecer por la derecha.

Se usa la operacion modulo (

El proceso general es:

1. crear arreglos temporales para cada fila;
2. calcular la nueva posicion de cada pixel;
3. copiar los datos a la fila temporal;
4. reemplazar la fila original por la nueva.

6.2. Movimiento vertical

`trasladar_vertical` hace lo mismo, pero en el eje Y.

En este caso se usan matrices temporales completas porque el desplazamiento se aplica por filas.

El proceso es:

1. crear matrices temporales;
2. calcular la nueva fila de cada pixel;
3. copiar la informacion en la posicion correspondiente;
4. reemplazar la matriz original por la transformada.

7. Rotacion

Cuando se llama a `rotate()`, internamente se ejecuta `rotar_90()`.

Esta funcion rota la imagen 90 grados en sentido antihorario.

7.1. Como funciona

1. Se crean matrices temporales.
2. Se aplica la transformacion de indices necesaria para rotar una matriz bidimensional.
3. Se copia el resultado a las matrices principales.
4. Se libera la memoria temporal.

En esencia, es geometria aplicada sobre indices de arreglos.

8. Historial de acciones

Para implementar *undo* y *redo*, se usan estructuras de la STL de C++:

- `std::stack<NodoHistorial>pila_deshacer`
- `std::stack<NodoHistorial>pila_rehacer`
- `std::queue<NodoHistorial>cola_ejecucion`

Cada accion se guarda en un `NodoHistorial`, que contiene:

- `tipo`: operacion realizada;
- `magnitud`: cantidad de pixeles movidos, o cero si solo se rota.

Los tipos estan definidos en el `enum TipoAccion`:

- `MOVER_IZQ`
- `MOVER_DER`
- `MOVER_ARRIBA`
- `MOVER_ABAJO`
- `ROTAR`

9. Registro de acciones

Cada vez que se ejecuta un movimiento nuevo, la clase hace lo siguiente:

1. guarda la accion en la pila de deshacer;
2. la agrega a la cola de ejecucion;
3. actualiza la ultima accion registrada;
4. limpia la pila de rehacer si era necesario.

Esto es importante porque, si se hace una nueva accion despues de deshacer algo, el historial de rehacer deja de ser valido.

10. Deshacer (undo)

`undo()` revierte la ultima accion guardada en la pila de deshacer.

10.1. Proceso

1. Se revisa si la pila esta vacia.
2. Se toma la accion de la parte superior.
3. Esa accion se guarda en la pila de rehacer.
4. Se aplica la operacion inversa.

10.2. Inversos usados

- `MOVER_DER` se deshace con movimiento a la izquierda.
- `MOVER_IZQ` se deshace con movimiento a la derecha.
- `MOVER_ABAJO` se deshace con movimiento hacia arriba.
- `MOVER_ARRIBA` se deshace con movimiento hacia abajo.
- `ROTAR` se deshace con tres rotaciones antihorarias, que equivalen a una rotacion horaria.

11. Rehacer (redo)

`redo()` recupera una accion que habia sido deshecha.

11.1. Proceso

1. Se revisa si la pila de rehacer esta vacia.
2. Se toma la accion guardada en esa pila.
3. Se vuelve a colocar en la pila de deshacer.
4. Se ejecuta nuevamente la misma transformacion.

En resumen, `redo()` vuelve a aplicar la operacion que ya habia sido revertida.

12. Repetir la ultima accion (repeat)

`repeat()` ejecuta otra vez la accion mas reciente.

A diferencia de `undo()`, esta funcion no modifica el historial de las pilas; solo usa la variable `ultima_accion`.

12.1. Ejemplo

Si la ultima accion fue `move_left(20)` y luego se llama a `repeat()`, la imagen se movera otros 20 pixeles a la izquierda.

13. Repetir toda la secuencia (`repeat_all`)

Esta funcion permite reconstruir toda la secuencia desde el estado inicial. `repeat_all()` realiza los siguientes pasos:

1. restaura la imagen inicial usando `base_*`;
2. copia la cola de ejecucion;
3. recorre cada accion en orden;
4. aplica cada transformacion una por una;
5. guarda un PNG por cada paso.

13.1. Resultado

Se generan archivos con nombres como:

- `repeat_all_00.png`
- `repeat_all_01.png`
- `repeat_all_02.png`

Esto permite ver la evolucion completa de la secuencia de transformaciones.

14. Generacion de archivos PNG

La funcion `draw(const char* nombre)` convierte la imagen actual en un archivo PNG.

14.1. Proceso

1. Reserva un arreglo lineal de bytes RGB en el *heap*.
2. Recorre la imagen y copia rojo, verde y azul en orden.
3. Crea la carpeta `imagenes` si no existe.
4. Abre el archivo de salida.
5. Llama a `svpng()` para escribir el PNG.
6. Libera la memoria temporal.

El archivo generado siempre se guarda dentro de la carpeta `imagenes/`.

15. Flujo completo de `test.cpp`

El archivo `test.cpp` funciona como una demostracion del programa.

15.1. Paso a paso

1. Limpia la carpeta de imagenes anteriores.
2. Crea un objeto `moving_image`.
3. Ejecuta 5 movimientos a la izquierda y guarda cada resultado.
4. Ejecuta 5 movimientos a la derecha y guarda cada resultado.
5. Ejecuta 5 movimientos hacia arriba y guarda cada resultado.
6. Ejecuta 5 movimientos hacia abajo y guarda cada resultado.
7. Ejecuta 4 rotaciones y guarda cada resultado.
8. Prueba `undo()` y `redo()`.
9. Prueba `repeat()`.
10. Ejecuta `repeat_all()` para reconstruir toda la secuencia.

16. Estructuras de datos utilizadas

El programa usa estructuras lineales porque las acciones tienen un orden temporal claro.

- **Stack (pila):** sirve para *undo* y *redo*, ya que permite acceder a la ultima accion realizada.
- **Queue (cola):** sirve para guardar la secuencia completa y reproducirla en el mismo orden en que ocurrio.

Estas estructuras hacen que el manejo del historial sea ordenado y facil de seguir.

17. Idea general del comportamiento

En resumen, el programa hace lo siguiente:

- carga una imagen base en memoria;
- modifica sus pixeles con movimientos y rotaciones;
- guarda cada estado en disco como PNG;
- permite deshacer, rehacer, repetir y reproducir toda la secuencia;
- usa pilas y colas para administrar el historial de acciones.

18. Observacion final

Todo el trabajo se realiza sobre matrices de pixeles en memoria. No se usa una libreria de ventanas ni una interfaz grafica interactiva. Por eso el programa genera archivos PNG por separado, en lugar de mostrar la imagen en una aplicacion visual.

19. Resumen corto

La clase `moving_image` es un sistema de transformacion de imagenes en C++. Permite mover, rotar, deshacer, rehacer y repetir acciones, manteniendo un historial con pilas y colas. Gracias a esto, el programa no solo modifica la imagen, sino que tambien permite reconstruir toda la secuencia de operaciones.